

Cours de Programmation en C – EI-2I

Travaux Dirigés

TD2

Exercice 1 – Tant que

Écrire un programme qui lit des entiers au clavier tant qu'il n'y a pas d'entiers négatifs et en affiche le plus grand et le plus petit

1. en utilisant une boucle `while`
2. en utilisant une boucle `do...while`
3. en utilisant une boucle `for`

Solution:

1.

```
int mini,maxi; /* pour stocker les valeurs mini et maxi en cours */
int v;          /* valeur courante */

/* obtenir une 1ere valeur */
scanf( "%d", &v);
mini=v;
maxi=v;

/* on itère tant que c'est positif */
while (v>0)
{
    /* mise-à-jour du mini et maxi */
    if (mini>v)
        mini = v;
    if (maxi<v)
        maxi = v;
    /* on obtient la nouvelle valeur */
    scanf( "%d", &v);
}
```
2.

```
int mini,maxi; /* pour stocker les valeurs mini et maxi en cours */
int v;          /* valeur courante */

/* valeur initiale de mini et maxi */
mini = INT_MAX;
maxi = INT_MIN; /* définis dans "limits.h", à inclure

/* on itère tant que c'est positif */
do
{
    /* on obtient la nouvelle valeur */
    scanf( "%d", &v);

    /* mise-à-jour du mini et maxi; on est obligé de tester de nouveau v */
    if (v>0)
    {
        if (mini>v)
            mini = v;
        if (maxi<v)
            maxi = v;
    }
} while (v>0);
```
3.

```
int mini,maxi; /* pour stocker les valeurs mini et maxi en cours */
int v;          /* valeur courante */

/* on itère tant que c'est positif */
for( scanf( "%d", &v), mini=maxi=v; v>0; scanf( "%d", &v))
{
    /* mise-à-jour du mini et maxi */
    if (mini>v)
        mini = v;
    if (maxi<v)
        maxi = v;
}
```

La boucle `while` est ici la plus adaptée.

Exercice 2 – Boucle for

On suppose $a \leq b$.

1. Écrire une boucle qui affiche les entiers de a (inclus) à b (inclus)
2. Écrire une boucle qui affiche les entiers de a (inclus) à b (exclus)
3. Écrire une boucle qui affiche les entiers de a à b par ordre décroissant (a et b inclus)

Solution:

```
1.  int i;
    for( i=a; i<=b; i++)
    {
        printf("%d", i);
    }

2.  int i;
    for( i=a; i<b; i++)
    {
        printf("%d", i);
    }

3.  int i;
    for( i=b; i>=a; i--)
    {
        printf("%d", i);
    }
```

Les itérations les plus difficiles sont la première et la dernière itération. À chaque fois qu'une boucle est conçue, il faut se poser les questions:

- Est-ce que je démarre bien de la bonne valeur ?
- À quel moment je sors de ma boucle? Est-ce la bonne valeur ?

D'une manière générale, il faut toujours vérifier que la condition qui permet de sortir de la boucle peut **évoluer** et s'assurer que cette condition deviendra `true` à un moment donné.

Exercice 3 – Conversion hexadécimale

On souhaite écrire un programme qui lit un nombre hexadécimal comme une suite de caractères, vérifie sa validité et effectue la conversion décimale. L'itération s'arrêtera quand on lira un caractère ne représentant pas un chiffre hexadécimal, et on calculera la valeur au fur et à mesure.

- Écrire en pseudo-code l'algorithme
- Écrire le code C correspondant

Solution:

- Algorithme

```
lire(c)
```

```
Tant que c est valide faire
// on traite la valeur lue
nb <- nb*16
```

```
si '0' <= c <= '9' alors
nb <- nb + (c - '0')
sinon si 'a' <= c <= 'f' alors
nb <- nb + (c - 'a' + 10)
sinon
nb <- nb + (c - 'A' + 10)
fini
```

```
// on obtient la nouvelle valeur
lire(c)
FinTantQue
```

puis (approche descendante)

"c est valide" s'écrit (c>='0' et c<='9') ou (c>='a' et c<='f') ou (c>='A' et c<='F')

- le code:

```
/*=====
 *
 * oO conversion Hexa->Décimal Oo
 *
 *=====
 *
 * File : conversionHexa.c
 * Date : 10sept 09
 * Author : Hilaire Thibault
 * Honteusement repris depuis "La programmation C et C++",
 * Badouel/Khaled, ed. Hermes
 *
 *=====
 *
 * Notions:
 * - itérations dans un tant que
 * - structure "lire, tant que pas fini, traiter, lire suivant"
 * - conversion char -> int
 */

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int nb = 0;
    char c;

    /* obtenir le 1er caractère */
    c = getchar();

    /* traiter jusqu'à caractère non valide */
    while ( ((c>='0') && (c<='9')) || ((c>='a') && (c<='f')) || ((c>='A') && (c<='F')) )
    {
        /* mise-à-jour de nb avec le nouveau caractère*/
        nb = nb*16;
        if ( (c>='0') && (c<='9') )
            nb = nb + (c - '0');
        else if ( (c>='a') && (c<='f') )
            nb = nb + (c - 'a' + 10);
        else
            nb = nb + (c - 'A' + 10);

        /* obtenir le caractère suivant */
        c = getchar();
    }

    /* afficher le résultat */
    printf("En_décimal_\u:_%d\n", nb);

    return EXIT_SUCCESS;
}
```

Exercice 4 – Fractions égyptiennes

Toute fraction peut s'écrire comme une somme de fractions ayant 1 comme numérateur. Cette décomposition est

appelée décomposition en fractions égyptiennes. En voici un exemple :

$$\frac{7}{8} = \frac{1}{2} + \frac{1}{3} + \frac{1}{24}$$

Écrire un algorithme prenant en entrée le numérateur a et le dénominateur b d'une fraction et affiche sa décomposition en fractions égyptiennes (cette décomposition n'est pas unique).

Solution:

Une 1ère solution naïve

```
/* Fraction Egyptiennes
1ère méthode "naive" où on enlève tous les 1/i avec i qui va de 1 à ...
*/

#include <stdio.h>

int main()
{
    int num,den;    /* numérateur et dénominateur */
    int i = 1;

    /* saisie de num et den, strictement positifs */
    num=den=0;
    while (num<=0 || den<=0 || num>den)
    {
        printf("Saisissez le numérateur et le dénominateur (avec num>den, num>0 et den>0)\n");
        scanf("%d%d", &num, &den);
    }

    /* on itere pour enlever des 1/i */
    printf("%d/%d=", num, den);
    while (num>0)
    {
        /* si num/den>=1/i */
        if (num*i >= den)
        {
            printf("1/%d", i);

            /* on enlève 1/i à num/den */
            num = num*i - den;
            den = den * i;

            /* affichage du "+" si num/den n'est pas nul */
            if (num)
                printf("+");
        }
        i++;
    }

    printf("\n");

    return 0;
}
```

Une autre basée sur un algorithme glouton de Fibonacci (1202) (redécouvert par Sylvester en 1880).

"soustraire à la fraction donnée la plus grande fraction égyptienne possible, répéter l'opération avec la nouvelle fraction et ainsi de suite..."

```
/* Fraction Egyptiennes
2ème méthode, algo glouton de Fibonacci (1202) (redécouvert par Sylvester en 1880)
"soustraire à la fraction donnée la plus grande fraction égyptienne possible, répéter l'opération avec la nouvelle fraction et ainsi de suite..."
*/

#include <stdio.h>
#include <math.h>

int main()
{
    int num,den;    /* numérateur et dénominateur */
    int i;

    /* saisie de num et den, strictement positifs */
}
```